

Outputs, Redirection & Errors

This PDF will focus on teaching you how to deal with output, and how to *put your output* into a file; and how to **control** the output result.

We will make a **script** called "ORGANIZE_DOWNLOADS" to *organize* our downloads folder.

- BELOW, are the *commands* we'll use in this script:
 - REMEMBER; **scripts** *MUST* start with `#!/usr/bin/env bash` on the first line.
 - REMEMBER; scripts should be in the *scripts folder* shown in the last PDF.

> `echo` THIS, is a simple command to *spit out the text* you give it to the screen.

```
echo "Organizing Downloads Folder"
organizing downloads folder
```

For the purpose of the script, make the next line go to Downloads folder.
`cd ~/Downloads`.

Now the key new part, **output redirection**.

> `mkdir -p archives 2>/dev/null` The left side is the same as usual, makes a folder called archives.

On the RIGHT; it says `2>/dev/null`, `2` is just **errors** from a command, (the error here would be the folder existing), the `>` means put whats on the left on the right, and `/dev/null` is a folder that *deletes anything inside itself*.

- In effect, this creates the folder if it doesn't exist, and if it **DOES**, it carries on.

Let's do this for a couple other folders.

```
mkdir -p archives 2>/dev/null
mkdir -p images 2>/dev/null
mkdir -p workdocuments 2>/dev/null
```

Now, let's actually *move* these files.

```
mv *.jpg *.png *.jpeg images/ 2>/dev/null
mv *.pptx *.docx *.xlsx workdocuments/ 2>/dev/null
mv *.tar *.gz *.zip archives/ 2>/dev/null
```

The *left side* is just taking any file with those extensions, into the given folder *we made above* (or didn't).

Again `2>/dev/null` takes errors, (the error here would be, there's NONE of the specified file type to move)

Now *one last thing left to teach*, let's say; we want to have a logging_text file that shows when we ran the command.

- You may think to use the ">" to put the *date command* into a **log file**

Problem is, that **OVERWRITES** old data.

- The `>>` takes what on the left, and **ADDS** it to the text file on the right, *leaving alone the older lines*.

```
touch organizationlog.txt 2>/dev/null
date >> organizationlog.txt
```

> `touch organizationlog.txt 2>/dev/null` Creates the log files, and *if it already exists* throws away the error.

> `date >> organization.txt` ADDS to the log file with the date.

You can add some finishing touches, by adding more text by saying "Done"

Now, you should have all of these lines on the picture below in your file, as per the last PDF, **make the file executable**.

> `chmod u+x ORGANIZE_DOWNLOADS` *THIS* makes it **runnable**.

```
#!/usr/bin/env bash
echo "Organizing Downloads Folder"

cd ~/Downloads

mkdir -p archives 2>/dev/null
mkdir -p images 2>/dev/null
mkdir -p workdocuments 2>/dev/null

mv *.jpg *.png *.jpeg images/ 2>/dev/null
mv *.pptx *.docx *.xlsx workdocuments/ 2>/dev/null
mv *.tar *.gz *.zip archives/ 2>/dev/null

touch organizationlog.txt 2>/dev/null
date >> organizationlog.txt

echo "Done"
```

With that, you JUST made your first **bash script**. Next we'll handle more complex scripts *with variables*.